Comparative study of symbolic execution tools applied to vulnerability detection

André Cardoso
School of Technology
Polytechnic Institute of Cávado and Ave
Barcelos, Portugal
a18848@alunos.ipca.pt

Nuno Lopes
2AI - School of Technology, IPCA
Barcelos, Portugal
LASI - Associate Laboratory of Intelligent Systems
Guimarães, Portugal
0000-0001-8897-5061

***Abstract*—The security of an application showed to be a critical aspect of software development, and one way to test for vulnerabilities is through symbolic execution.**

This paper will present a comparative study of two symbolic execution tools, KLEE and Owi, applied to vulnerability detection in software applications. After experimenting with both tools, and analysing the results, it's noted better performance and more comprehensive results with KLEE.

***Index Terms*—Symbolic Execution, Application Security Testing, Static Code Analysis, KLEE, Owi**

I. INTRODUCTION

The security of an application showed its first impacts around the beginning of the century with the increasing popularity of the WWW (World Wide Web) and the enablement of user-fuelled websites (like social media). This rapid evolution paired with the common lack of knowledge of the users but also of the developers caused the birth of Denial of Service and Cross-Site Scripting attacks among others [1].

As such, many tools to find vulnerabilities in an automated fashion have been developed and distributed, each with their own set of pros and cons. The tools in question can be categorized as static or dynamic, and each on their own use different technics to accomplish different results. SAST (Static Application Security Testing) ultimately relies on some variety of pattern matching, data-flow analysis, and symbolic execution [2],

The objective is a comparison of two symbolic execution tools and see how they compare in terms of performance and their ability to detect vulnerabilities. To achieve this comparison, the tools must run on projects that are preferably coded in the same language, but if these tools do not support the same set of languages, the projects should have similar architecture or business logic/intent (e.g.: online stores, accounting services, ...). These tools will also be compared on how they can be tuned in order to achieve better results.

For a better overview of how the comparison will be performed, here follows a set of questions and metrics used to measure the tools:

- Time to test a project
- CPU usage while testing a project
- Memory usage while testing a project
- Analysis of vulnerabilities detection (Youden's Index)
- Complexity of the vulnerabilities detected
- Ability to find vulnerabilities out-of-the-box
- Ability to find vulnerabilities with proper manipulation
- Effort of tuning the tool

In sum, the expected output is a clear comparison of tools that can assist developers and software companies in keeping code secure proactively and explain what differs between them and why one should pick one over the other(s). These is how we intend to answer the following research questions:

- **Which tool is more effective at finding vulnerabilities?**
- **Which tool is more efficient in terms of performance?**

Section II presents a brief introduction to the theme at hand and its relevance to today's software dilemma of Application Security Testing. Then, section III introduces some of the available tools for symbolic execution, which will be selected for testing, comparison, and discussion in terms off what they offer, how they work and how to interact with them. In section IV the test environment is introduced, then the test cases, which metrics are collected, and in the end results are presented and analysed. Finally, in section V we conclude with a summary of the paper.

II. RELATED WORK

Expanding on SAST, most vulnerabilities are due to developer's lack of awareness and/or bad programming practices [3] in such a way that OWASP Foundation [4] defines it "as part of a Code Review", a "method of computer program debugging that is done by examining the code without executing the program", and also a good technic to help find: "Syntax violations", "Security vulnerabilities", "Programming errors", "Coding standard violations", and "Undefined values".

A. Symbolic Execution

Inside SAST's set of technics, introduced by [5], we find symbolic execution engines which can help protect systems that range from modern SoC (System-on-Chip) designs and hardware Trojans [6], to binary-level security analysis such as malware comprehension [7] and even a symbiosis between symbolic and concrete execution [8]. All these tools share a common requirement: any instruction set ranging between machine code to source code. These tools excel by mimicking the

execution of the analysed software, controlling each value the input influences in the form of symbolic values or variables.

However, a more in-depth look at symbolic execution unveils some obstacles, such as path explosion, constraint solving, environment modelling and others. These symbolic values are allowed to be anything at a first stage, and its value is deciphered further down in the execution path, reducing the possibilities at every branch the execution encounters. The branching happens after performing constraint solving, a very time expensive operation especially when the tested source is of high complexity [9].

Constraint solving occurs at every moment where a variable value impacts the flow of execution or even once a symbolic variable takes part in any arithmetic computation. This escalates once these computations become of a non-linear nature, such as non-linear expressions and non-linear floating-point computations [10].

Path explosion is one concern found in many high performance problems: the input is sufficiently complex for any sort of resource - be it time, memory, or even processing capacity - to escalate tremendously and becoming unreasonable in a standard environment. When it comes to symbolic execution, a new path is encountered every time a constraint is solved; from that point on, that constraint is a path condition. To better understand this concept, here's an example using the tool *KLEE* according to [11, p. 3]:

> "When KLEE detects an error or when a path reaches an exit call, KLEE solves the current path's constraints (called its path condition) to produce a test case that will follow the same path when rerun on an unmodified version of the checked program (e.g., compiled with gcc)."

Environment modelling is another very common problem in software analysis techniques given how much software development relies on third-party libraries today. This problem escalates whenever these dependencies are packaged as binaries, or the sources are 'super-optimized', the paths exploration and consequently the constraint solving may fail which may result in false positives or false negatives [11].

B. Symbolic Execution Tools

There are a couple of tools available that perform symbolic execution in numerous ways with different focus and strengths. A good agglomerate of these tools is maintained in a GitHub repository owned by Luckow [12].

- ***SPF (Symbolic PathFinder)*** SPF is an extension for the JPF (Java PathFinder), a 'model checking framework for Java™ bytecode programs' [13, 14].
- ***Jalangi2*** Jalangi2 is a framework for analysis of JavaScript programs, designed to work on browser applications [15].
- ***SymJS*** SymJS explores Javascript applications and is packed with a symbolic execution engine and also 'an automatic event explorer for Web pages' [16].
- ***Cloud9*** Cloud9 was designed with the intent of reducing the 'resource-intensive ... nature of high-quality software testing' [17].
- ***Kite*** Kite is a Proof-of-Concept tool that tackles the symbolic execution problem using the CDSE (Conflict-Driven Symbolic Execution) algorithm which is able to dynamically reduce path exploration and is also capable of learning from earlier conflicts [18, 19].
- ***KLEE*** KLEE is perhaps the most relevant tool to address in the LLVM field considering how both Kite and Cloud9 were developed on top of the work acomplished by KLEE. KLEE becomes dynamic through its outside environment interaction features [11].
- ***Owi*** Owi presents promising work specially by their attempt to integrate with the WASM developer lifecycle with subcommands that allow for other features useful for the everyday tasks like formatting of files and validation of modules but also supports symbolic execution [20].

KLEE and Owi are the selected tools for this comparison, given multiple factors such as their wide language support, their impact on Web Applications, and the availability of sources and documentation. Other tools such as SPF and Jalangi2 are also interesting but they are focused on specific languages and their learning curve is quite steep, especially for beginners. SymJS, Cloud9, and Kite are also interesting but they lack sources and documentation, making it very hard or even impossible to use them for this comparison.

III. FEATURE DESCRIPTION

This section focuses on describing KLEE and Owi set of features (subsections III-A and III-B), finishing with a comparison between both tools, presented in section III-C.

A. KLEE

KLEE explores the LLVM infrastructure rather successfully and with speed; however, this engine bottlenecks quite considerably when it is applied on larger scale projects. This is due to its space management, given how it tries to keep all relevant information about the current states. It is also unable to handle dynamically generated code, concurrent execution and struggles with modelling network and peripherals, although most peers share these weaknesses [21]. The LLVM project supports many different languages like Haskell, Rust and Python, but our test cases will focus on C/C++ programs [22].

LLVM has been referenced a few times in this document, but a brief explanation is in order. LLVM is not an acronym, but the full name of the project. Despite the name, it has nothing to do with virtual machines. LLVM is "a collection of modular and reusable compiler and toolchain technologies". It is also praised for its versatility, flexibility, and reusability. For more information about LLVM, please refer to their website [22].

1) Features: KLEE is packaged with multiple CLIs that contribute to the user experience, but the main command that runs the symbolic execution engine is `klee`. Another relevant

command is `ktest-tool`, which describes the generated test cases (`.ktest` files) in a human-readable format.

2) Instrumentation: The KLEE web page contains a tutorials section to help get familiarized with the tool. These tutorials contain samples and instructions on how to properly set up a C/C++ source file, let's call it code instrumentation, and how to run and analyse the results [23, 24].

The first, and most basic example, shows how one can mark a variable as symbolic by using the `klee_make_symbolic()` function This function takes 3 arguments; the first two will define a memory range in which the variable information is stored, the third defines a human-readable object name that identifies the symbolic variable in the generated test case files.

B. Owi

Owi is a toolkit that aggregates functionality to assist WebAssembly developers, one of which enables the compilation of C code to WASM and running their symbolic interpreter on top of it [25].

WebAssembly is a portable binary instruction format for executable programs, designed to be a compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications. WebAssembly is designed to be fast, efficient, and secure, making it an ideal choice for running complex applications in web browsers and other environments [26, 27].

1) Features: Owi being a toolchain designed for the development of WASM, it provides a set of subcommands but this study will focus on the ones related to symbolic execution, which is the '*sym*' subcommand. To be able to use the symbolic interpreter (`owi sym`), we need to also install a restraint solver. Owi supports any Smt.ml supported solver, but the default solver is Z3.

2) Instrumentation: Unlike KLEE, that uses the variable address only and ignores the type, Owi needs to be told the symbolic variable type, through the available functions like `owi_i32()` for 32-bit integers, `owi_f32()` for 32-bit floating-point values, `owi_char()` for character values and `owi_bool()` for boolean values. Each of these functions also has variants for other types, such as 64-bit integers and floating-point values.

The symbols are identified by order of creation - if we define all positions of an array as symbolic, each position symbolic identifier would correspond to their array position, but if any other symbol is defined before the identifiers would shift by the number of predefined variables.

To run Owi on a simple sample we must include the Owi library in the source code so it is possible to find Owi function implementations and generate a valid WebAssembly module.

C. Feature Comparison

Not only it is important to understand how well a tool performs, but also how flexible it can be in the different scenarios it may face. Table I summarizes the comparison, and is divided into 3 sections.

TABLE I: Feature comparison

*F*eature	KLEE	Owi
Installation speed	5 (2)	2
Installation complexity	1 (4)	3
Ease-of-use (begginer)	4	2
Ease-of-use (experienced)	4	4
Heuristics support	Larger	Reduced
Heuristics definition	Hardcoded	Hardcoded
Supports E-ACSL	NO	YES
SAT/SMT solvers support	Larger	Reduced

The first section of Table I describes the installation process and ease-of-use of each tool. Installation is scored in two dimensions: speed and complexity. KLEE supports a quick start using Docker, which is very fast to set up but may not be the best option for everyone. The recommended installation process is more complex and time-consuming, which is reflected in the scores. Owi doesn't offer a docker image.

The second section of Table I describes features and heuristics supported by each tool; heuristics refers to software vulnerabilities/errors, if they are supported by the tools and how these heuristics are defined.

The last section of Table I summarizes how KLEE and Owi compare in terms of supporting SAT and SMT solvers. While SAT solvers handle *boolean satisfiability*, SMT expand on this concept into general formulas[28]. After looking at both KLEE and Owi documentation, we can see that KLEE supports more solvers overall.

IV. EXPERIENCES AND RESULTS

In this section, multiple experiences using both KLEE and Owi tools will be described and analysed. We will start by describing the test environment in section IV-A, and then proceed to summarize the experiments ran in section IV-B. The metrics used to evaluate the tools will be presented in section IV-C, and finally a summary of all results and an analysis will be presented in section IV-D. Each experience will showcase their capabilities and limitations.

A. Test Environment

To increase the results' credibility, all tests were run on the same machine, therefore sharing the same resources. The specifications of such machine consisted in a 12th Gen Intel Core processor (i7-1255U) of 10 cores (and a total of 12 threads) with a max clock speed of 4.70 GHz, and 20 GB (4 + 16) of DDR4 3200 MHz RAM.

This is the parent system setup with a Windows 11 installation; however, the test environment was created in a WSL environment within this machine which shares 4 cores and 8 GB of RAM available.

For simplicity of testing, all samples were organized by folders and subfolders: the parent folder aggregates by sample and child folders aggregates files by tool, KLEE or Owi.

B. Test Samples

To compare the tools we will run them in 9 samples. The samples 1 and 2 were extracted from KLEE's online

documentation [23, 24] and samples 3 through 5 from Owi's GitHub repository [20]. Samples 6 through 9 were recovered from ADALogics blog [29]. All samples were adapted with necessary changes to ensure compatibility with both tools. They cover, in that order, the following vulnerabilities:

- Global Buffer Overflow
- Stack-based Buffer Overflow
- NULL pointer dereference
- Use after `free`

We can look at sample 7 as an example of how KLEE and Owi compare. Stack-based Buffer Overflow vulnerabilities look at scoped and dynamically allocated variables like the one on line 8 of listing 1. To run KLEE and Owi on this sample, the sources were instrumented as shown on Listing 2 and Listing 3, respectively.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <klee/klee.h>
5
6  int test2(char *buf)
7  {
8    int *dyn_mem_arr = malloc(sizeof(int) * 10);
9    dyn_mem_arr[0] = 0;
10   dyn_mem_arr[1] = 1;
11   dyn_mem_arr[2] = 2;
12   dyn_mem_arr[3] = 3;
13   dyn_mem_arr[4] = 4;
14   dyn_mem_arr[5] = 5;
15   dyn_mem_arr[6] = 6;
16   dyn_mem_arr[7] = 7;
17   dyn_mem_arr[8] = 8;
18   dyn_mem_arr[9] = 9;
19
20   int i = 5;
21   if (strcmp(buf, "BUG!") == 0)
22   {
23     i += 20;
24   }
25
26   int return_value = dyn_mem_arr[i];
27   free(dyn_mem_arr);
28   return return_value;
29 }
```

Listing 1: Sample 7 - test2 function code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <klee/klee.h>
5
6  int test2(char *buf){/* ... */}
7
8  int main(int argc, char const *argv[])
9  {
10   char arr[10];
11   klee_make_symbolic(arr, 10, "arr");
12   klee_assume(arr[9] == '\0');
13
14   test2(arr);
15   return 0;
16 }
```

Listing 2: Sample 7 - KLEE instrumentation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <owi.h>
5
6  int test2(char *buf){/* ... */}
7
8  int main(int argc, char const *argv[])
9  {
10   char arr[10];
11   for (int i = 0; i < 10; i++)
12   {
13     arr[i] = owi_char();
14   }
15   owi_assume(arr[9] == '\0');
16
17   test2(arr);
18   return 0;
19 }
```

Listing 3: Sample 7 - Owi instrumentation

C. Metrics

This section discusses **Accuracy** and **Performance** metrics such as True Positives (**TP**), False Positives (**FP**), True Negatives (**TN**), False Negatives (**FN**), and Youden Index (**J**) also known as Youden's J statistic:

Accuracy Metrics

- **TP**: Number of expected vulnerabilities reported.
- **TN**: Number of unexpected vulnerabilities not reported.
- **FP**: Number of unexpected vulnerabilities reported.
- **FN**: Number of expected vulnerabilities not reported.
- **Youden Index**: The accuracy metric used in OWASP Benchmark [30] is the following

$$J = TPR - FPR = \left(\frac{TP}{TP + FN}\right) - \left(\frac{FP}{TN + FP}\right)$$

TPR - True Positive Rate
FPR - False Positive Rate

When reading the Youden Index, the higher the value, the better the tool is at detecting vulnerabilities. OWASP Benchmark also provides a Youden Index interpretation guide that uses both TPR and FPR values [30].

Performance Metrics

- **Paths traced**: Branches of execution created to navigate.
- **CPU (%)**: The CPU used by the process, in percentage.
- **Memory (KB)**: The 'maximum resident set size' (very similar to peak memory) used by the process, in kilobytes.
- **Time (s)**: The execution time of the process, in seconds.

Accuracy metrics were extracted by manually inspecting each scan output, and performance metrics such as CPU, memory and time of execution were measured using the GNU `time` program. "**Paths traced**" was also extracted manually similarly to accuracy metrics. There is also the "**Success**" metric that indicates whether the sample was successfully analysed by the test tool, without regard for results.

D. Results and Analysis

After executing both engines on all test samples and extracting the results from each of them, Tables II and III were

TABLE II: KLEE accuracy metrics

#[a]	TP	TN	FP	FN
1	0	1	0	0
2	2	0	0	0
3	1	0	0	0
4	0	1	0	0
5	0	0	1	1
6	1	0	0	0
7	1	0	0	0
8	1	0	0	0
9	1	0	0	0

[a]Sample number

TABLE III: Owi accuracy metrics

#[a]	TP	TN	FP	FN
1	0	1	0	0
2	0	0	0	1
3	1	0	0	0
4	0	1	0	0
5	1	0	0	0
6	0	0	0	1
7	0	0	0	1
8	0	0	0	1
9	0	0	0	1

[a]Sample number

created. With these metrics properly formatted, calculating the Youden Index for both tools is easier, higher is better.

It is apparent how KLEE detected much more vulnerabilities than Owi, which plays in its favour, and the calculations on equations (1) and (2) reflect this statement, KLEE scores approximately 54% while Owi stands on 29% of bug detection accuracy.

If we look at the Youden Index interpretation guide from OWASP Benchmark [30], we can see that KLEE scores highly (87.5%) in the *y* axis (TPR), while Owi scores poorly (28.5%). Both tools score low on the *x* axis (FPR), with KLEE at 33.3% and Owi at 0%. This means that KLEE is very good at detecting vulnerabilities, but also has a higher rate of false positives, while Owi is not very good at detecting vulnerabilities, but has a low rate of false positives. A larger set of samples would be needed to draw more concrete conclusions.

$$J = (\frac{7}{7+1})^{*} - (\frac{1}{2+1})^{**} \approx (0.875) - (0.333) \approx 0.54 \quad (1)$$

KLEE Youden Index

$$J = (\frac{2}{2+5})^{*} - (\frac{0}{2+0})^{**} \approx (0.285) - (0) \approx 0.29 \quad (2)$$

Owi Youden Index

* TPR - True Positive Rate

** FPR - False Positive Rate

By running KLEE and Owi on the same set of samples, some differences are visible in their outputs and in their performances (see Section IV-D). KLEE was overall slower, with the only exception being the primes example, but was also the example that consumed less CPU but also Memory on all samples.

Unfortunately, Owi does not seem to output information regarding the travelled states, even after looking at debug logs (which depict execution instructions, but nothing was found to support a number of traversed paths).

An example of how the performance between these tools differ is with sample 3, a test where KLEE traces 306 paths. KLEE reported results consuming nearly half the CPU resources and approximately 73% of the memory usage and 35% of the time spent, that Owi required to run and return no results on the sample.

TABLE IV: KLEE performance metrics

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	3	99	98'156	0.29
2	YES	6'675	101	139'680	12.93
3	YES	305	93	99'152	0.72
4	YES	99	98	99'504	2.93
5	YES	1	81	98'036	0.39
6	YES	5	102	98'920	0.30
7	YES	5	87	97'536	0.37
8	YES	5	96	98'248	0.33
9	YES	5	96	99'236	0.32

[a]Sample number
[b]The best out of five runs

TABLE V: Owi performance metrics

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	N/A	108	110'000	0.05
2	YES	N/A	200	725'892	901.51
3	YES	N/A	199	134'928	2.10
4	YES	N/A	204	145'676	9.90
5	YES	N/A	205	151'800	11.13
6	YES	N/A	116	120'172	0.06
7	YES	N/A	115	119'172	0.06
8	YES	N/A	118	118'220	0.05
9	YES	N/A	123	121'040	0.05

[a]Sample number
[b]The best out of five runs

Sample 2 shows an even greater difference; KLEE spends nearly half the CPU and 19% of the memory usage when compared to Owi, and does that in 1% of the time it took Owi to analyse the same source. This scenario might be a clear indicator of Owi limitations.

V. CONCLUSION

The nine tests performed allowed us to establish how each tool compares in terms of speed, and CPU and memory usage.

The metrics collected helped conclude that KLEE was overall superior both in performance and in its bug-finding capabilities. Despite being superior at finding vulnerabilities, KLEE still scores poorly in the OWASP Benchmark scoring table (54%) [30], mainly due to its False Positive Rate, but the real culprit of this result is the size of the test sample.

In conclusion, this work aims at making symbolic execution a familiar concept in the life of a software developer; despite the investigated tools being oriented to specific development platforms, the examined samples were not, proving we can still compile programs to an architecture that is not the intended for a 'production environment' and have good testing mechanisms.

VI. ACKNOWLEDGMENTS

XXX

REFERENCES

[1] A. Hoffman, *Web Application Security*. O'Reilly Media, Inc., 2024.

[2] R. Croft, D. Newlands, Z. Chen, and A. M. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," *International Symposium on Empirical Software Engineering and Measurement*, 10 2021. [Online]. Available: http://eprints.whiterose.ac.uk/95628/

[3] M. Alenezi and Y. Javed, "Open source web application security: A static analysis approach," *IEEE*, pp. 1–5, 2016.

[4] OWASP Foundation. OWASP DevSecOps Guideline - v-0.2 | 2-a - Static Application Security Testing. (accessed Feb. 29, 2024). [Online]. Available: https://owasp.org/www-project-devsecops-guideline/latest/02a-Static-Application-Security-Testing

[5] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, pp. 385–394, 7 1976.

[6] A. Vafaei, N. Hooten, M. Tehranipoor, and F. Farahmandi, "Symba: Symbolic execution at c-level for hardware trojan activation," *Proceedings - International Test Conference*, pp. 223–232, 2021.

[7] R. David, S. Bardin, T. D. Ta, J. Feist, L. Mounier, M. L. Potet, and J. Y. Marion, "Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis," *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER 2016*, vol. 1, pp. 653–656, 5 2016.

[8] F. Gritti, L. Fontana, E. Gustafson, F. Pagani, A. Continella, C. Kruegel, and G. Vigna, "Symbion: Interleaving symbolic with concrete execution," *2020 IEEE Conference on Communications and Network Security, CNS 2020*, 6 2020.

[9] G. Zhang, Z. Chen, Z. Shuai, Y. Zhang, and J. Wang, "Synergizing symbolic execution and fuzzing by function-level selective symbolization," *Proceedings - Asia-Pacific Software Engineering Conference, APSEC*, vol. 2022-December, pp. 328–337, 12 2022.

[10] D. Kroening and O. Strichman, *Decision Procedures - An Algorithmic Point of View*, 1st ed. Springer Berlin, Heidelberg, 4 2008.

[11] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, 2008. [Online]. Available: https://purl.stanford.edu/fx501rc2898

[12] K. Luckow. ksluckow/awesome-symbolic-execution. (accessed Nov. 25, 2024). [Online]. Available: https://github.com/ksluckow/awesome-symbolic-execution

[13] Java Pathfinder. javapathfinder/jpf-symbc: Symbolic PathFinder. (accessed Jan. 22, 2025). [Online]. Available: https://github.com/javapathfinder/jpf-symbc

[14] ——. javapathfinder/jpf-core Wiki. (accessed Jan. 22, 2025). [Online]. Available: https://github.com/javapathfinder/jpf-core/wiki

[15] Samsung. Samsung/jalangi2: Dynamic analysis framework for JavaScript. (accessed Jan. 23, 2025). [Online]. Available: https://github.com/Samsung/jalangi2

[16] G. Li, E. Andreasen, and I. Ghosh, "SymJS: automatic symbolic testing of JavaScript web applications," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. New York, NY, USA: ACM, Nov. 2014.

[17] Dependable Systems Lab. Cloud9 - Automated Software Testing at Scale. (accessed Jan. 22, 2025). [Online]. Available: https://dslab.epfl.ch/research/cloud9/

[18] C. G. do Val, "Conflict-driven symbolic execution," Ph.D. dissertation, University of British Columbia, 2014.

[19] ——. Kite: A conflict-driven symbolic execution tool. (accessed Jan. 22, 2025). [Online]. Available: https://www.cs.ubc.ca/labs/isd/Projects/Kite/

[20] Formal Security for Web Technologies. OCamlPro/owi: WebAssembly Swissknife & cross-language bugfinder. (accessed Dec. 24, 2024). [Online]. Available: https://github.com/OCamlPro/owi

[21] N. M. d. Sabino, "Automatic vulnerability detection: Using compressed execution traces to guide symbolic execution," Master's thesis, Instituto Superior Técnico, 2019.

[22] LLVM Project. The LLVM Compiler Infrastructure Project. (accessed Jan. 22, 2025). [Online]. Available: https://llvm.org/

[23] The KLEE Team. Tutorial One · KLEE. (accessed Dec. 23, 2024). [Online]. Available: https://klee-se.org/tutorials/testing-function

[24] ——. Tutorial Two · KLEE. (accessed Dec. 23, 2024). [Online]. Available: https://klee-se.org/tutorials/testing-regex

[25] L. Andrès, F. Marques, A. Carcano, P. Chambart, J. F. F. dos Santos, and J.-C. Filliâtre, "Owi: Performant parallel symbolic execution made easy, an application to webassembly," *The Art, Science, and Engineering of Programming*, vol. 9, 10 2024. [Online]. Available: https://hal.science/hal-04627413

[26] WebAssembly. WebAssembly. (accessed Jan. 10, 2026). [Online]. Available: https://webassembly.org

[27] ——. Use Cases - WebAssembly. (accessed Jan. 10, 2026). [Online]. Available: https://webassembly.org/docs/use-cases/

[28] RBC Borealis. Tutorial #9: SAT Solvers I: Introduction and applications. (accessed Jan. 21, 2025). [Online]. Available: https://rbcborealis.com/research-blogs/tutorial-9-sat-solvers-i-introduction-and-applications/

[29] ADALogics. Symbolic execution with KLEE: From installation and introduction to bug-finding in open source software. (accessed Dec. 17, 2024). [Online]. Available: https://adalogics.com/blog/symbolic-execution-with-klee

[30] OWASP Foundation. OWASP Benchmark | OWASP Foundation. (accessed Jan. 9, 2025). [Online]. Available: https://owasp.org/www-project-benchmark/